

# PostgreSQL Mastery Roadmap — Zero to Staff Engineer

*A visual, phase-by-phase progression from zero SQL knowledge to staff/principal-level PostgreSQL expertise.*

## Table of Contents

- [1. How to Read This Roadmap](#)
- [2. Complete Visual Roadmap](#)
- [3. Phase 1 — Beginner](#)
- [4. Phase 2 — SQL Practitioner](#)
- [5. Phase 3 — PostgreSQL Developer](#)
- [6. Phase 4 — Senior Engineer](#)
- [7. Phase 5 — Staff / Principal Engineer](#)
- [8. Interview Readiness by Phase](#)
- [9. Skill Dependency Tree](#)
- [10. Progression Checklist](#)

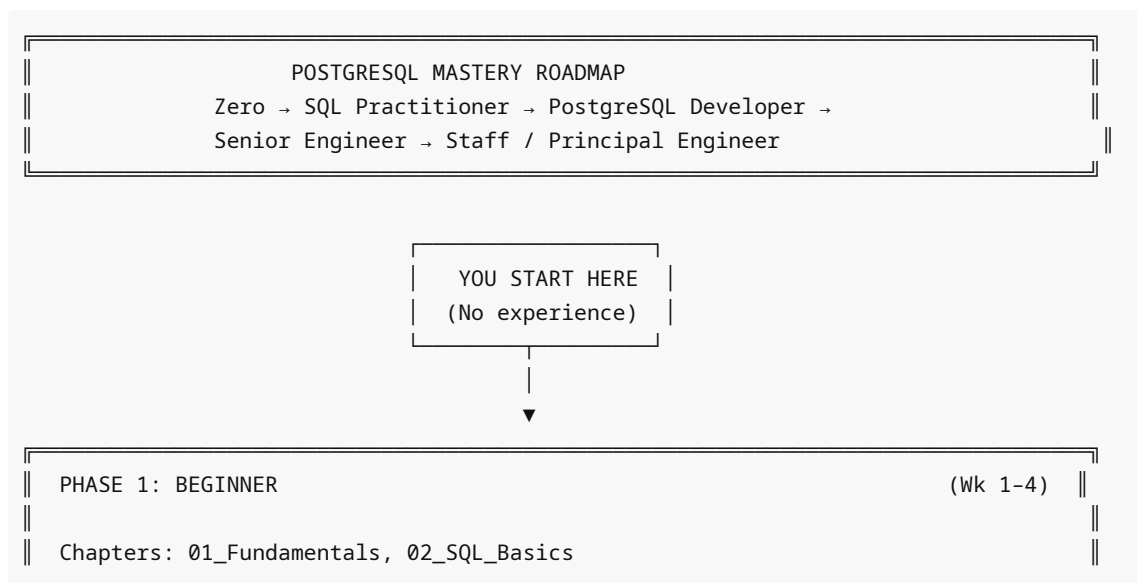
## 1. How to Read This Roadmap

Each phase in this roadmap represents a stable plateau of competence — a level where you can operate independently and productively in a professional setting. Phases are not just about what you know; they are about what you can **do** without looking things up.

**Phase gates** are checkpoints. Do not advance to the next phase until you can pass the phase gate without notes or hints. These gates are hard but fair — they represent the real minimum bar for each level.

**Interview readiness** at each phase tells you roughly which types of roles you can compete for. This is calibrated against actual interview standards at mid-to-large tech companies.

## 2. Complete Visual Roadmap



Skills Unlocked:

- ♦ Relational model – tables, rows, columns, keys
- ♦ SELECT, WHERE, ORDER BY, LIMIT
- ♦ All JOIN types (INNER, LEFT, RIGHT, FULL, CROSS)
- ♦ GROUP BY, HAVING, aggregate functions
- ♦ INSERT, UPDATE, DELETE, UPSERT
- ♦ NULL semantics and safe NULL handling
- ♦ psql basics, connecting to PostgreSQL

Phase Gate: Write a 4-table JOIN with GROUP BY, HAVING, and CASE without hints

Weeks 4-8 ▼

PHASE 2: SQL PRACTITIONER

(Wk 5-8)

Chapters: 03\_Intermediate\_SQL, 04\_Advanced\_SQL

Skills Unlocked:

- ♦ Common Table Expressions (CTEs) – simple and chained
- ♦ Correlated subqueries and EXISTS
- ♦ Window functions: ROW\_NUMBER, RANK, DENSE\_RANK, NTILE
- ♦ Window functions: LEAD, LAG, FIRST\_VALUE, LAST\_VALUE
- ♦ Window frames: ROWS vs RANGE, UNBOUNDED PRECEDING
- ♦ Running totals, moving averages, cumulative distributions
- ♦ Recursive CTEs for hierarchical/graph data
- ♦ LATERAL joins for row-wise correlated computation
- ♦ Pivot/unpivot, conditional aggregation, gaps and islands
- ♦ Date/time arithmetic, generate\_series

Phase Gate: Solve 10 window function problems timed – avg < 4 min each

Phase Gate: Write a recursive CTE for an org chart (manager-reports)

Weeks 9-14 ▼

PHASE 3: POSTGRESQL DEVELOPER

(Wk 9-14)

Chapters: 05-09 (PostgreSQL Core through Transactions)

Skills Unlocked:

- ♦ PostgreSQL type system: numeric, text, UUID, arrays, JSONB, ranges
- ♦ JSONB: operators, path queries, GIN indexes, performance
- ♦ Schema design: normalization (1NF-3NF-BCNF), constraints, ERD
- ♦ Primary key strategies: serial, BIGSERIAL, UUID, ULID
- ♦ All index types: B-Tree, Hash, GIN, GiST, BRIN, partial, expression
- ♦ Multi-column index ordering and covering indexes (INCLUDE)
- ♦ EXPLAIN and EXPLAIN ANALYZE – reading all node types
- ♦ Seq Scan, Index Scan, Index Only Scan, Bitmap Heap Scan
- ♦ Hash Join, Merge Join, Nested Loop – when each is used
- ♦ pg\_stat\_statements – identifying slow queries

- ♦ ACID properties and what they mean in practice
- ♦ MVCC: how PostgreSQL implements concurrent reads/writes
- ♦ Transaction isolation levels: Read Committed, Repeatable Read, Serializable
- ♦ Row locking: SELECT FOR UPDATE, FOR SHARE, NOWAIT, SKIP LOCKED
- ♦ Deadlock detection, pg\_locks, pg\_blocking\_pids

Projects: Project 1 (E-commerce DB), Project 2 (Event Tracking)

Phase Gate: Explain MVCC – xmin, xmax, snapshot, visibility rules

Phase Gate: Read a real EXPLAIN ANALYZE plan and identify the bottleneck

Phase Gate: Design a normalized schema for a SaaS app from scratch

Weeks 15-22 ▼

#### PHASE 4: SENIOR POSTGRESQL ENGINEER

(Wk 15-22)

Chapters: 10-17 (Internals through Backend/Data Engineering)

Skills Unlocked:

- ♦ WAL (Write-Ahead Log): structure, LSN, WAL writer, pg\_wal directory
- ♦ Heap file layout: page header, item pointers, tuple header, xmin/xmax
- ♦ Checkpoint process: BGWriter, checkpoint\_completion\_target
- ♦ Shared buffer pool: buffer eviction (clock-sweep algorithm)
- ♦ TOAST: threshold, storage strategies, detoasting cost
- ♦ VACUUM: dead tuple cleanup, freeze, visibility map update, FSM update
- ♦ Autovacuum: trigger thresholds, tuning per-table, monitoring lag
- ♦ Bloat: causes, measurement, pg\_bloat\_check, VACUUM FULL vs pg\_repack
- ♦ JSONB full-text: tsvector, tsquery, GIN indexes, ranking
- ♦ Foreign Data Wrappers: postgres\_fdw, query pushdown, limitations
- ♦ Logical decoding, replication slots, pg\_logical\_slot\_get\_changes
- ♦ Table partitioning: range, list, hash, partition pruning
- ♦ postgresql.conf tuning: memory, WAL, checkpoints, connections
- ♦ Autovacuum deep tuning: cost delay, cost limit, per-table settings
- ♦ PgBouncer: session vs transaction vs statement mode, pool sizing
- ♦ Monitoring: pg\_stat\_\*, prometheus-postgres\_exporter, Grafana dashboards
- ♦ Streaming replication: primary/standby setup, replication slots
- ♦ Logical replication: publications, subscriptions, conflicts, filtering
- ♦ Row-level security (RLS): policies, USING, WITH CHECK, bypass
- ♦ Roles and privileges: GRANT hierarchy, SECURITY DEFINER, pgaudit
- ♦ pg\_hba.conf: authentication methods, scram-sha-256
- ♦ Backup: pg\_dump, pg\_basebackup, WAL archiving
- ♦ PITR: continuous archiving, restore\_command, recovery target
- ♦ Zero-downtime migrations: CREATE INDEX CONCURRENTLY, column add/drop
- ♦ LISTEN/NOTIFY for lightweight async messaging
- ♦ Advisory locks for distributed coordination

Projects: Project 3 (Multi-Tenant SaaS), Project 4 (Financial Ledger)

Phase Gate: Deploy streaming replication (primary + 2 standbys) from scratch

Phase Gate: Implement PITR, corrupt a table, restore to 1 minute before

Phase Gate: Implement RLS for a 3-tenant system, prove isolation

Phase Gate: Tune autovacuum for a 1B-row table with high update rate

Weeks 23-30 ▼

PHASE 5: STAFF / PRINCIPAL ENGINEER

(Wk 23-30)

Chapters: 18-25 (Architecture, Interviews, System Design, Capstone)

Skills Unlocked:

- ♦ Patroni HA: architecture, etcd/consul DCS, automatic failover, pg\_rewind
- ♦ Patroni REST API, patronictl, switchover vs failover
- ♦ pgBackRest: stanza configuration, parallel backup, restore, WAL archiving
- ♦ Replication lag analysis: replication\_slots, pg\_stat\_replication, alerting
- ♦ Multi-datacenter replication topology design
- ♦ Sharding strategies: application-level, Citus, FDW-based, when to shard
- ♦ Connection pool architecture: HAProxy + PgBouncer + Patroni
- ♦ Capacity planning: connections, disk I/O, memory, CPU modeling
- ♦ Upgrade strategies: pg\_upgrade, logical replication upgrade, blue-green
- ♦ PostgreSQL at scale: GitLab (1TB+), Shopify, Notion case studies
- ♦ Cost modeling: hardware selection, cloud instance sizing
- ♦ Cross-database consistency patterns (saga, two-phase commit)
- ♦ CDC architecture: Debezium + Kafka + PostgreSQL logical replication
- ♦ Compliance: GDPR data deletion in PostgreSQL, SOC2 audit trail
- ♦ PostgreSQL extension development: custom index access methods
- ♦ Leading technical decisions: trade-off articulation, RFC writing
- ♦ Mentoring junior engineers through PostgreSQL concepts
- ♦ Interview skill: explaining any PostgreSQL concept at any depth level

Projects: Project 5 (FTS Engine), Project 6 (HA Lab), Capstone

Phase Gate: Lead a 45-min system design interview for a ride-sharing DB

Phase Gate: Complete the Capstone and present architecture decisions

Phase Gate: Write a technical RFC for a PostgreSQL migration or upgrade

STAFF ENGINEER  
/ PRINCIPAL  
DATABASE EXPERT

### 3. Phase 1 — Beginner

Duration: 4 weeks | Chapters: 01, 02

#### Skills to Unlock

Beginner Skills Tree

```
|— Relational Model
|   |— Tables, rows, columns
|   |— Primary keys and foreign keys
|   |— Relationships: one-to-one, one-to-many, many-to-many
|   |— Cardinality and entity modeling
|
|— Basic Query Writing
|   |— SELECT with column selection
|   |— WHERE with comparison operators
|   |— BETWEEN, IN, LIKE, IS NULL
|   |— AND, OR, NOT logical operators
|   |— ORDER BY (single and multi-column)
|   |— LIMIT and OFFSET
|
|— Data Modification
|   |— INSERT (single row, multi-row)
|   |— UPDATE with WHERE
|   |— DELETE with WHERE
|   |— TRUNCATE
|   |— INSERT ... ON CONFLICT (UPSERT)
|
|— Joins and Set Operations
|   |— INNER JOIN
|   |— LEFT/RIGHT OUTER JOIN
|   |— FULL OUTER JOIN
|   |— CROSS JOIN
|   |— SELF JOIN
|   |— UNION / UNION ALL
|   |— INTERSECT / EXCEPT
|
|— Aggregation
|   |— COUNT, SUM, AVG, MIN, MAX
|   |— GROUP BY with single and multiple columns
|   |— HAVING for group-level filtering
|   |— DISTINCT and COUNT(DISTINCT)
|   |— CASE WHEN expressions
```

## Chapters to Read

- See [01\\_Fundamentals](#) — relational model, SQL history, psql setup
- See [02\\_SQL\\_Basics](#) — all query patterns above

## Projects to Complete

- None required, but complete all in-chapter exercises

## Interview Readiness

At the end of Phase 1, you can pass: **junior analyst / junior data role SQL screening**. A basic 30-minute "write a query" screen. Not yet ready for engineering roles.

---

## 4. Phase 2 — SQL Practitioner

**Duration:** 4 weeks | **Chapters:** 03, 04

## Skills to Unlock

### SQL Practitioner Skills Tree

- ├─ CTEs and Subqueries
  - ├─ Scalar subqueries (returns 1 value)
  - ├─ Row subqueries (returns 1 row)
  - ├─ Table subqueries (returns many rows)
  - ├─ Correlated subqueries (reference outer query)
  - ├─ EXISTS and NOT EXISTS
  - ├─ CTE with WITH clause
  - ├─ Multiple CTEs in sequence
  - └─ CTEs for readability vs performance trade-off
- ├─ Window Functions
  - ├─ OVER() clause basics
  - ├─ PARTITION BY for group-within-group
  - ├─ ORDER BY within window
  - ├─ ROW\_NUMBER() – unique sequential rank
  - ├─ RANK() – gaps in rank for ties
  - ├─ DENSE\_RANK() – no gaps for ties
  - ├─ NTILE(n) – percentile bucketing
  - ├─ LEAD() and LAG() – access adjacent rows
  - ├─ FIRST\_VALUE() / LAST\_VALUE()
  - ├─ NTH\_VALUE()
  - ├─ Window aggregates: SUM, AVG, COUNT OVER
  - ├─ ROWS BETWEEN frames
  - ├─ RANGE BETWEEN frames
  - └─ GROUPS BETWEEN frames (PG 11+)
- ├─ Advanced Patterns
  - ├─ Recursive CTEs (WITH RECURSIVE)
  - ├─ LATERAL joins
  - ├─ Pivot via conditional aggregation
  - ├─ Unpivot via UNION ALL
  - ├─ Gaps and islands pattern
  - ├─ Consecutive sequence detection
  - ├─ Date spine with generate\_series
  - ├─ String aggregation: string\_agg, array\_agg
  - └─ FILTER clause on aggregates
- └─ Date and Time
  - ├─ DATE, TIME, TIMESTAMP, TIMESTAMPTZ
  - ├─ INTERVAL arithmetic
  - ├─ date\_trunc() for bucketing
  - ├─ date\_part() / EXTRACT()
  - ├─ AT TIME ZONE
  - └─ generate\_series for date ranges

## Chapters to Read

- See `03_Intermediate_SQL` — CTEs, window functions, subqueries
- See `04_Advanced_SQL` — recursive CTEs, lateral, advanced patterns

## Projects to Complete

- Complete the cohort retention analysis exercise in `22_Projects` (mini)
- Complete the "Analytics Challenge" problems in `20_Interview_Tasks`

## Interview Readiness

At the end of Phase 2, you can pass: **mid-level data analyst SQL round**, **junior data engineer SQL screen**, most analytics engineer take-home tests.

---

# 5. Phase 3 — PostgreSQL Developer

**Duration:** 6 weeks | **Chapters:** 05–09

## Skills to Unlock

### PostgreSQL Developer Skills Tree

```
|
|— PostgreSQL Type System
|   |— Numeric: INTEGER, BIGINT, NUMERIC, REAL, DOUBLE PRECISION
|   |— Text: TEXT, VARCHAR, CHAR, CITEXT
|   |— Temporal: DATE, TIMESTAMP, TIMESTAMPTZ, INTERVAL
|   |— UUID and ULID
|   |— BOOLEAN
|   |— Arrays: declaration, operators, GIN indexing
|   |— JSONB vs JSON: storage, operators, indexing
|   |— Range types: int4range, tstzrange, numrange
|   |— Network types: inet, cidr, macaddr
|   |— Geometric types: point, box, circle, polygon
|   |— Type casting: ::, CAST(), implicit vs explicit
|
|— Schema Design
|   |— First Normal Form (1NF): atomic values, no repeating groups
|   |— Second Normal Form (2NF): no partial dependencies
|   |— Third Normal Form (3NF): no transitive dependencies
|   |— BCNF: every determinant is a candidate key
|   |— Denormalization: when and why
|   |— Primary key: SERIAL, BIGSERIAL, UUID, gen_random_uuid()
|   |— Foreign key: ON DELETE, ON UPDATE actions
|   |— UNIQUE constraints (single and multi-column)
|   |— CHECK constraints
|   |— NOT NULL constraints
|   |— EXCLUDE constraints (for range overlap prevention)
|   |— Partial UNIQUE indexes as conditional constraints
|
|— Indexes
|   |— B-Tree: structure, pages, height, splits
|   |— Hash: point lookups, equality only
```

- └─ GIN: inverted index, arrays, JSONB, full-text
- └─ GiST: generalized search tree, ranges, geometric
- └─ BRIN: block range, time-series, physical order
- └─ Partial indexes: index only matching rows
- └─ Expression indexes: index on transformed values
- └─ Multi-column indexes: column ordering matters
- └─ Covering indexes: INCLUDE clause
- └─ Index Only Scan: when it's possible
- └─ CREATE INDEX CONCURRENTLY
- └─ Query Optimization
  - └─ EXPLAIN: estimated costs and row counts
  - └─ EXPLAIN ANALYZE: actual timing and rows
  - └─ EXPLAIN (ANALYZE, BUFFERS): cache hit analysis
  - └─ Plan nodes: Seq Scan, Index Scan, Index Only Scan
  - └─ Plan nodes: Bitmap Index Scan + Bitmap Heap Scan
  - └─ Plan nodes: Hash Join, Merge Join, Nested Loop
  - └─ Plan nodes: Sort, Hash, Aggregate, GroupAggregate
  - └─ Cost model: seq\_page\_cost, random\_page\_cost, cpu\_tuple\_cost
  - └─ Statistics: pg\_stats, pg\_statistic, n\_distinct, correlation
  - └─ ANALYZE: collecting statistics
  - └─ pg\_stat\_statements: identifying top queries by total time
  - └─ SET enable\_seqscan = off for testing
- └─ Transactions and Concurrency
  - └─ BEGIN, COMMIT, ROLLBACK
  - └─ SAVEPOINT and ROLLBACK TO SAVEPOINT
  - └─ ACID: what each property means in practice
  - └─ MVCC: snapshot isolation, xmin, xmax, cmin, cmax
  - └─ Snapshot: transaction ID, visibility rules
  - └─ Read Committed: snapshot per statement
  - └─ Repeatable Read: snapshot per transaction
  - └─ Serializable: SSI (Serializable Snapshot Isolation)
  - └─ Read phenomena: dirty read, non-repeatable, phantom
  - └─ Table-level locks: SHARE, EXCLUSIVE, ACCESS EXCLUSIVE
  - └─ Row-level locks: FOR UPDATE, FOR SHARE, FOR NO KEY UPDATE
  - └─ NOWAIT: error if lock unavailable
  - └─ SKIP LOCKED: skip locked rows (queue processing)
  - └─ Advisory locks: pg\_try\_advisory\_lock, pg\_advisory\_lock
  - └─ Deadlock: detection, logging, avoidance patterns
  - └─ pg\_locks and pg\_blocking\_pids

## Chapters to Read

- See 05\_PostgreSQL\_Core
- See 06\_Database\_Design
- See 07\_Indexes
- See 08\_Query\_Optimization
- See 09\_Transactions\_Concurrency

## Projects to Complete

- 22\_Projects/project\_01 — E-Commerce Analytics Database



- 22\_Projects/project\_02 — Real-Time Event Tracking System

## Interview Readiness

At the end of Phase 3, you can pass: **mid-to-senior backend engineer SQL rounds, junior DBA technical screens, data engineer technical interviews** at most companies. Cannot yet discuss internals or production operations at depth.

---

## 6. Phase 4 — Senior Engineer

**Duration:** 8 weeks | **Chapters:** 10-17

### Skills to Unlock

#### Senior Engineer Skills Tree

- PostgreSQL Internals
  - Heap file: pages, item IDs, tuple headers
  - Page layout: pd\_lsn, pd\_lower, pd\_upper, pd\_special
  - Tuple header: xmin, xmax, ctid, natts, infomask
  - Dead tuples: why they accumulate, cost
  - VACUUM: mark dead tuples, update FSM, update VM
  - VACUUM FULL: table rewrite, lock required
  - Freeze: wraparound prevention, xid\_age
  - Visibility map: 2 bits per page (all-visible, all-frozen)
  - Free space map: tracks available space per page
  - TOAST: threshold (2KB), storage strategies
  - TOAST strategies: PLAIN, EXTERNAL, EXTENDED, MAIN
  - Shared buffer pool: pages in memory, dirty pages
  - Buffer eviction: clock-sweep algorithm
  - WAL: write-ahead, crash safety, LSN
  - WAL records: resource manager, operation type
  - WAL writer: continuous flushing to disk
  - Checkpoint: dirty page flush, WAL segment recycling
  - BGWriter: proactive dirty page flushing
  - Crash recovery: replay WAL from last checkpoint
- Production Configuration
  - shared\_buffers: 25% of RAM
  - effective\_cache\_size: total OS + PG cache
  - work\_mem: per sort/hash operation (not per connection)
  - maintenance\_work\_mem: VACUUM, CREATE INDEX
  - max\_connections: relationship to connection pool
  - wal\_level: minimal, replica, logical
  - checkpoint\_completion\_target: spread checkpoint I/O
  - max\_wal\_size: WAL growth budget between checkpoints
  - autovacuum\_vacuum\_cost\_delay: throttle VACUUM I/O
  - autovacuum\_vacuum\_scale\_factor: trigger threshold
  - random\_page\_cost: set to 1.1 for SSD
  - effective\_io\_concurrency: parallel I/O for bitmaps
  - max\_parallel\_workers\_per\_gather: parallel queries

- └─ Replication
  - └─ Physical replication: block-level WAL streaming
  - └─ pg\_hba.conf replication entries
  - └─ Primary\_conninfo in recovery.conf (PG 11) / postgresql.conf
  - └─ Replication slots: prevent WAL deletion
  - └─ pg\_stat\_replication: lag monitoring
  - └─ pg\_replication\_slot\_advance(): advance confirmed\_lsn
  - └─ Hot standby: read queries on replica
  - └─ hot\_standby\_feedback: prevent query cancellation
  - └─ Logical replication: row-level, selective
  - └─ Publication: what tables/columns/operations to publish
  - └─ Subscription: connect to publication, apply changes
  - └─ Conflict detection and resolution
  - └─ wal2json / pgoutput decoders
- └─ Backup and Recovery
  - └─ pg\_dump: logical dump, schema/data/both
  - └─ Custom format: pg\_dump -Fc, pg\_restore --jobs
  - └─ pg\_basebackup: physical backup of data directory
  - └─ WAL archiving: archive\_command, archive\_status
  - └─ restore\_command for WAL retrieval
  - └─ recovery\_target\_time, \_lsn, \_xid, \_name
  - └─ promote: pg\_promote() or pg\_ctl promote
  - └─ pgBackRest: parallel, encrypted, incremental backups
  - └─ Backup verification: restore and checksum
- └─ Security
  - └─ Authentication: md5 vs scram-sha-256 (prefer SCRAM)
  - └─ pg\_hba.conf: host, address, database, user, method
  - └─ Role system: LOGIN, SUPERUSER, CREATEDB, CREATEROLE
  - └─ Schema privileges: USAGE, CREATE
  - └─ Table privileges: SELECT, INSERT, UPDATE, DELETE
  - └─ Column-level privileges
  - └─ Row-Level Security: ALTER TABLE ENABLE ROW LEVEL SECURITY
  - └─ RLS USING clause: filter rows on read
  - └─ RLS WITH CHECK: validate rows on write
  - └─ SECURITY DEFINER: run function as owner
  - └─ pgaudit: session audit, object audit
  - └─ SSL: ssl=on, ssl\_cert\_file, ssl\_key\_file
  - └─ Privilege escalation: how to prevent it

## Chapters to Read

- See 10\_PostgreSQL\_Internals
- See 11\_Advanced\_PostgreSQL
- See 12\_Production\_PostgreSQL
- See 13\_Replication\_HA
- See 14\_Security
- See 15\_Backup\_Recovery
- See 16\_PostgreSQL\_For\_Data\_Engineering
- See 17\_PostgreSQL\_For\_Backend\_Engineers

## Projects to Complete

- 22\_Projects/project\_03 — Multi-Tenant SaaS Platform
- 22\_Projects/project\_04 — Financial Transaction Ledger

## Interview Readiness

At the end of Phase 4, you can pass: **senior backend engineer full loop, senior data engineer full loop, senior DBA / database engineer full loop, L5/L6 system design components** involving databases.

---

## 7. Phase 5 — Staff / Principal Engineer

**Duration:** 8 weeks | **Chapters:** 18–25

### Skills to Unlock

#### Staff / Principal Skills Tree

```
|
|— High Availability Architecture
|   |— Patroni: etcd-backed consensus for leader election
|   |— patronictl: switchover, failover, list, history
|   |— DCS: etcd vs consul vs ZooKeeper trade-offs
|   |— pg_rewind: sync diverged standby after failover
|   |— Cascading replication for remote DCs
|   |— PgBouncer + HAProxy + Patroni: full stack
|   |— Split-brain prevention and fencing
|
|— Sharding and Scale-Out
|   |— When NOT to shard (most cases)
|   |— Application-level sharding: shard key selection
|   |— Citus: distributed PostgreSQL
|   |— FDW-based federation as pseudo-sharding
|   |— Hot partition problem and mitigation
|   |— Cross-shard queries: complexity and cost
|   |— Re-sharding: the nightmare you want to avoid
|
|— Database Engineering at Scale
|   |— Zero-downtime major version upgrades
|   |— pg_upgrade: binary upgrade, statistics rebuild
|   |— Logical replication upgrade (safest for large DBs)
|   |— Blue-green deployment for databases
|   |— Online schema changes with pg_repack
|   |— Capacity planning: IOPS, throughput, latency budgets
|   |— Cloud: RDS PostgreSQL vs Aurora PostgreSQL vs self-managed
|
|— System Design Mastery
|   |— Reading/writing ratios and their implications
|   |— Identifying when PostgreSQL is the wrong tool
|   |— PostgreSQL + Redis: caching layer design
|   |— PostgreSQL + Elasticsearch: search offloading
|   |— PostgreSQL + Kafka: CDC and event streaming
|   |— CQRS with PostgreSQL: read/write model separation
```

- └─ Two-phase commit: when and why to avoid it
- └─ Saga pattern as an alternative to distributed transactions
- └─ Leadership Skills
  - └─ RFC writing: database migration, infrastructure change
  - └─ Incident response: diagnosis playbook, postmortem
  - └─ Mentoring: explaining MVCC to a junior in 5 minutes
  - └─ Code review: catching N+1s, missing indexes, unsafe migrations
  - └─ Trade-off articulation: clarity, honesty, depth
  - └─ Cross-team influence: driving adoption of best practices

## Chapters to Read

- See 18\_Architecture\_Case\_Studies
- See 19\_Interview\_Questions (senior/staff sections)
- See 20\_Interview\_Tasks (advanced tasks)
- See 21\_System\_Design
- See 22\_Projects (projects 5 and 6)
- See 23\_Company\_Preparation
- See 25\_Capstone\_Program

## Projects to Complete

- 22\_Projects/project\_05 — PostgreSQL Full-Text Search Engine
- 22\_Projects/project\_06 — HA Replication Lab with Patroni
- 25\_Capstone\_Program — End-to-end production system

## Interview Readiness

At the end of Phase 5, you can lead: **staff/principal/distinguished engineer system design rounds, database architecture reviews, principal DBA interviews at FAANG.**

## 8. Interview Readiness by Phase

| Phase   | Junior (L3) | Mid (L4) | Senior (L5) | Staff (L6+) |
|---------|-------------|----------|-------------|-------------|
| Phase 1 | Partial     | No       | No          | No          |
| Phase 2 | Yes         | Partial  | No          | No          |
| Phase 3 | Yes         | Yes      | Partial     | No          |
| Phase 4 | Yes         | Yes      | Yes         | Partial     |
| Phase 5 | Yes         | Yes      | Yes         | Yes         |

**Partial** = Can pass some rounds but not all.

## 9. Skill Dependency Tree

- FOUNDATIONAL (learn first, cannot skip)
- └─ Relational model
  - └─ Basic SQL syntax

- └─ JOIN semantics
- └─ NULL handling

CORE SQL (required for all paths)

- └─ Aggregation and GROUP BY
- └─ Subqueries
- └─ CTEs
- └─ Window functions

POSTGRESQL DEVELOPER (required for engineering roles)

- └─ Type system
- └─ Index design
- └─ EXPLAIN ANALYZE
- └─ MVCC and transactions
- └─ Schema design

SENIOR (required for L5+ roles)

- └─ WAL and internals
- └─ VACUUM and autovacuum
- └─ Streaming replication
- └─ Backup/PITR
- └─ RLS and security
- └─ Production config tuning

STAFF (required for L6+ roles)

- └─ Patroni HA
- └─ Sharding design
- └─ Upgrade strategies
- └─ System design fluency
- └─ Technical leadership

---

## 10. Progression Checklist

Use this checklist to self-assess. Advance only when all items in a phase are checked.

### Phase 1 — Beginner

- ☐ Can explain what a relational database is and why we use one
- ☐ Can write a multi-table JOIN query from scratch
- ☐ Can use GROUP BY with HAVING correctly
- ☐ Can handle NULL values correctly in comparisons and aggregations
- ☐ Can use all set operators (UNION, INTERSECT, EXCEPT)
- ☐ psql: can connect, run queries, use meta-commands

### Phase 2 — SQL Practitioner

- ☐ Can write a non-trivial CTE chain (3+ CTEs)
- ☐ Can solve a correlated subquery problem
- ☐ Can calculate running total and 7-day moving average with window functions
- ☐ Can use LEAD/LAG to detect state changes

- ☐ Can write a recursive CTE for a tree structure
- ☐ Can use LATERAL join correctly

### Phase 3 — PostgreSQL Developer

- ☐ Can choose correct data type for any given column
- ☐ Can use JSONB operators and index JSONB fields
- ☐ Can design a normalized schema from a business description
- ☐ Can explain which index type to use and why for any scenario
- ☐ Can read and explain a 10-node EXPLAIN ANALYZE plan
- ☐ Can explain MVCC with xmin/xmax without notes
- ☐ Can implement SELECT FOR UPDATE correctly

### Phase 4 — Senior Engineer

- ☐ Can draw the WAL write path from query to disk
- ☐ Can explain VACUUM's effect on the visibility map
- ☐ Can tune autovacuum for a specific table workload
- ☐ Can set up streaming replication from scratch (primary + 2 replicas)
- ☐ Can perform a PITR restore and validate the result
- ☐ Can implement RLS policies for multi-tenant isolation
- ☐ Can configure PgBouncer in transaction mode
- ☐ Can identify and fix autovacuum lag

### Phase 5 — Staff / Principal Engineer

- ☐ Can deploy a Patroni cluster with automatic failover
- ☐ Can design a sharding strategy and explain the trade-offs
- ☐ Can lead a 45-minute system design session involving a database
- ☐ Can write a zero-downtime major version upgrade plan
- ☐ Can complete the Capstone project
- ☐ Can mentor a junior engineer through any Phase 1–3 concept